CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

CON TRỎ - CẤP PHÁT ĐỘNG - ỨNG DỤNG





Nội dung

1. Biến và vùng nhớ RAM
2. Toán tử & và *
3. Khái niệm con trỏ
4. Cú pháp khai báo
5. Con trỏ có giá trị NULL
6. Cấp phát bộ nhớ
7. Con trỏ và hàm
8. Con trỏ và chuỗi
9. Con trỏ trong mảng 1 chiều
10. Con trỏ và cấu trúc
11. Bài tập



Ý tưởng: Đặt nghi vấn

Xem chương trình sau

```
#include <iostream>
using namespace std;

int main()
{
    char c = 'F';
    int x = 10;
    float y = 1.8;

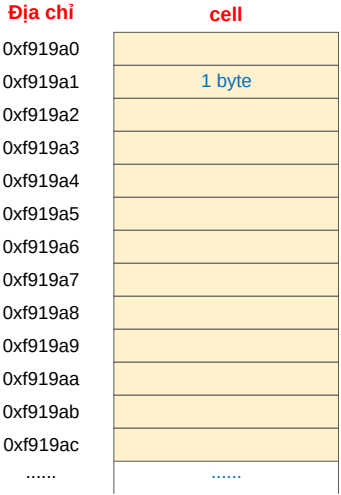
    return 0;
}
```

Các biến **c**, **x**, **y** được quản lý như thế nào?

Vùng nhớ RAM

Vùng nhớ RAM (Random Access Memory)

- RAM là vùng nhớ được sử dụng để lưu trữ các dữ liệu và chương trình đang hoạt động trong thời gian thực.
- Trong RAM có nhiều cell, mỗi cell trong RAM có một địa chỉ duy nhất để truy cập vào cell đó.
- Vùng nhớ RAM dùng để lưu trữ dữ liệu và dữ liệu có thể được truy cập bất kỳ vị trí nào trong RAM một cách ngẫu nhiên và nhanh chóng. Tức là có thể truy cập vào bất kỳ cell nào trong RAM một cách nhanh chóng.
- Nhưng, RAM là vùng nhớ tạm thời, tức là dữ liệu chỉ tồn tại trong khi máy tính đang hoạt động và bị xóa khi máy tính tắt.



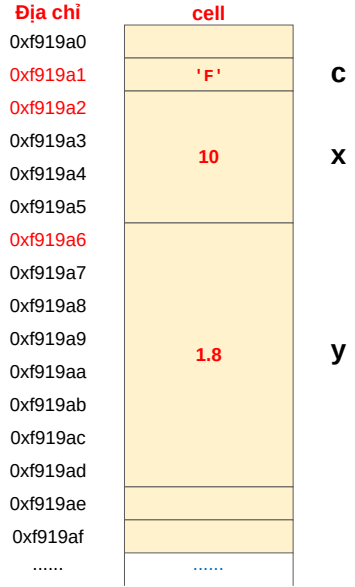
Vùng nhớ RAM

Biến và vùng nhớ RAM

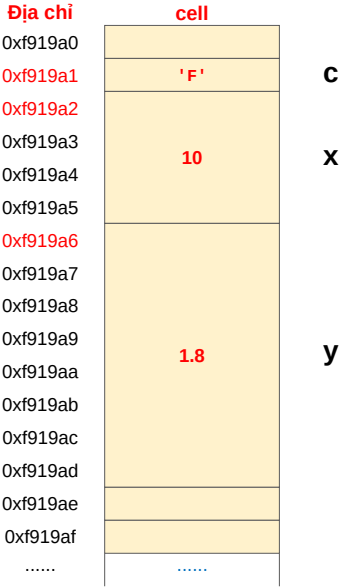
```
#include <iostream>
using namespace std;

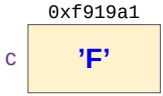
int main()
{
    char c = 'F';
    int x = 10;
    float y = 1.8;

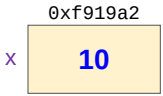
    return 0;
}
```

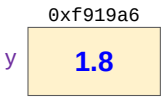


Cách vẽ gọn hơn









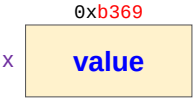


Toán tử & và *

Toán tử &

- Toán tử **&** dùng để lấy địa chỉ của biến.
- Phương pháp dùng:

```
&tên_biến
```



*Toán tử **

- Toán tử * được sử dụng để truy cập vào giá trị mà một con trỏ đang trỏ đến.
- (Toán tử * dùng để truy cập giá trị mà một địa chỉ đang quản lý.)
- Phương pháp dùng:

```
*địa_chỉ_biến
```

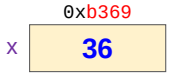
Toán tử & và *

Minh họa

```
#include <iostream>
using namespace std;
int main()
{
    int x = 36;

    cout << "Gia tri bien x = " <<  x << endl;
    cout << "Dia chi bien x : " << &x << endl;
    printf("Dia chi bien x : %p", &x); printf("\n");
    printf("Dia chi bien x : %x", &x);

    return 0;
}
```



Toán tử & và *

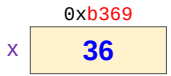
Minh họa

```
#include <iostream>
using namespace std;
int main()
{
    int x = 36;

    cout << "Gia tri bien x = " <<  x << endl;
    cout << "Dia chi bien x : " << &x << endl;

    cout << "Gia tri bien x = " <<  *(&x) << endl;

    return 0;
}
```





Khái niệm con trỏ (pointer)

Con trỏ - pointer

- Con trỏ là một kiểu dữ liệu đặc biệt dùng để lưu địa chỉ trong bộ nhớ.
- Con trỏ cho phép truy cập và thay đổi dữ liệu của biến được quản lý bởi một địa chỉ.
- Sử dụng con trỏ thì cần cẩn thận và chắc chắn sử dụng chính xác để tránh các vấn đề liên quan đến quản lý bộ nhớ.



Cú pháp khai báo

```
Kiểu_Dữ_Liệu* ten_con_tro;
```

Trong đó:

- `Kiểu_Dữ_Liệu` là kiểu dữ liệu của biến mà con trỏ sẽ trỏ tới. Ví dụ: int, float, char, double… hoặc kiểu dữ liệu khác.
- * gọi là toán tử con trỏ, đặt sau kiểu dữ liệu để khai báo là một con trỏ.
- `ten_con_tro` tên của con trỏ.

Ví dụ khai báo con trỏ

```
int* iP;
float* fP;

int *p1, *p2, *p3;

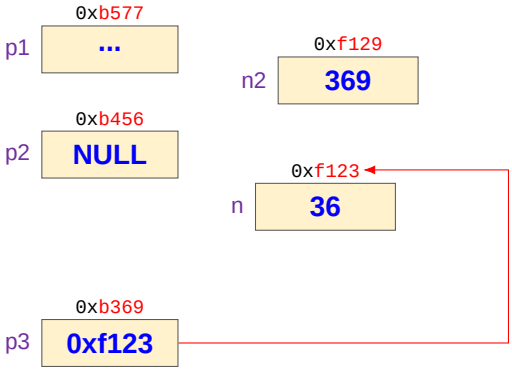

SinhVien* pSV;
ToaDo* pTD;
```

Con trỏ có giá trị NULL

- Khi khai báo kiểu con trỏ và không gán giá trị địa chỉ nào hết thì con trỏ đó chưa khởi tạo.
- Nếu gán giá trị **NULL** cho con trỏ thì gọi là **con trỏ NULL**, sẽ không trỏ vào địa chỉ của biến nào cả.
- Ví dụ:

```
int* p1;   //Con trỏ chưa khởi tạo
int* p2 = NULL; //Con trỏ NULL

int n = 36;
int* p3 = &n; //Con trỏ có địa chỉ
```



size hoặc sizeof()

- Để xác định kích thước của kiểu dữ liệu hoặc biến thì dùng size hoặc sizeof(). Đơn vị: byte
- Cú pháp:

```
sizeof(kiểu_dữ_liệu)
sizeof ten_bien
```

- Ví dụ:

```
double x = 9.6;
double* p;

cout << sizeof(int) << endl;          4
cout << sizeof(double) << endl;       8
cout << sizeof x << endl;             8
cout << sizeof p << endl;             4
```

Kích thước của một con trỏ trong C++ có thể khác nhau trên các nền tảng và trình biên dịch khác nhau.
32-bit (4 byte)
64-bit (8 byte)



Khái niệm cấp phát bộ nhớ

Cấp phát bộ bộ nhớ tĩnh (Static Memory Allocation)

- Được thực hiện bằng cách khai báo mảng hoặc biến với kích thước cụ thể tại thời điểm biên dịch.
- Các biến và mảng được cấp phát bộ nhớ tĩnh tồn tại trong suốt thời gian chạy chương trình và không thay đổi kích thước khi chạy.

Cấp phát động (Dynamic Memory Allocation)

- Cho phép cấp phát bộ nhớ trong quá trình chạy chương trình và giải phóng bộ nhớ khi không cần sử dụng nữa.



Phương pháp cấp nhát bộ nhớ

Ngôn ngữ	Cấp phát	Thu hồi – Giải phóng
C++	new	delete
C	malloc() calloc() hoặc realloc()	free()

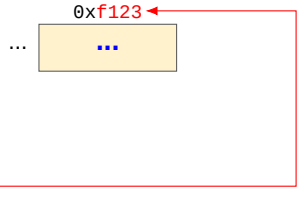
Cấp phát động

```
#include <iostream>
using namespace std;

int main()
{
    int* p = new int;

    if(p == NULL)
    {
        cout << "Cap phat khong thanh cong" << endl;
    }

    return 0;
}
```

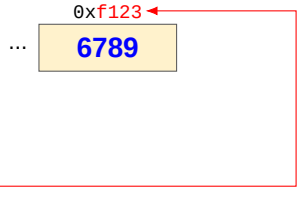


Cấp phát động

```
#include <iostream>
using namespace std;

int main()
{
    int* p = new int;

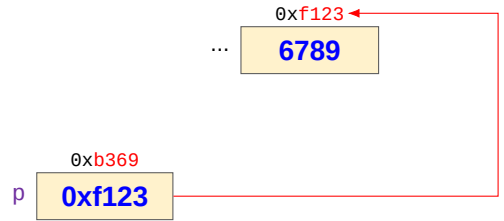
    if(p == NULL)
    {
        cout << "Cap phat không thanh cong" << endl;
    }
    *p = 6789;
    return 0;
}
```



Cú pháp

```
kiểu_dữ_liệu* ten_p = new kiểu_dữ_liệu(giá_trị);
```

- Ví dụ:




```
int* p = new int(6789);

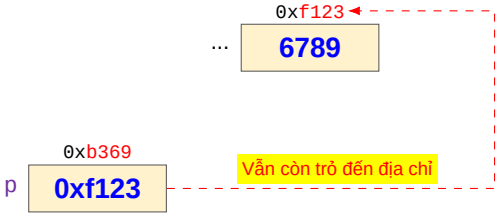
cout << *p; //6789
```

Thu hồi – Giải phóng

- Dùng toán tử **delete** để thu hồi vùng nhớ.
- Sau khi thu hồi vùng nhớ bằng delete thì con trỏ vẫn trỏ đến vùng nhớ.
- Có thể gọi tham chiếu nhưng kết quả không lường trước được.
- => Hiện tượng lạc con trỏ.

```
int* p = new int(6789);
cout << "Gia tri p = " << p << endl;
cout << *p << endl << endl; //6789

delete p;
cout << "Gia tri p = " << p << endl;
cout << *p << endl; //Giá trị ...
```



Kết quả

```
Gia tri p = 0xe919706789
Gia tri p = 0xe9197015302288
```

```
Gia tri p = 0xf419706789
Gia tri p = 0xf4197016023184
```

```
Gia tri p = 0x7819706789
Gia tri p = 0x7819707896720
```

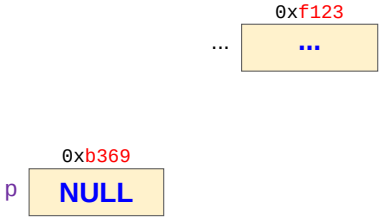
Thu hồi – Giải phóng

- Để tránh con trỏ bị thất lạc thì sau khi delete gán giá trị **NULL** cho con trỏ.
- Minh họa:

```
int* p = new int(6789);
cout << "Gia tri p = " << p << endl;
cout << *p << endl << endl; //6789

delete p;
p = NULL;

cout << "Gia tri p = " << p << endl;
//cout << *p << endl;
```





Con trỏ và hàm

- Con trỏ có thể dùng làm tham số của hàm hoặc kiểu dữ liệu của hàm.
- Ví dụ:

```
int* input()
{
    int* pi = new int;
    cout << "Nhap gia tri: ";
    cin >> *pi;

    return pi;
}

int main()
{
    int* p = input();
    cout << "Gia tri = " << *p << endl;
    return 0;
}
```



Con trỏ và hàm

- Ví dụ

```
void hoan_vi(int* a, int* b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}
int main()
{
    int x = 3, y = 6;
    cout << "Truoc hoan vi. x = " << x << "   y = " << y << endl;

    hoan_vi(&x, &y);
    cout << "Sau   hoan vi. x = " << x << "   y = " << y << endl;

    return 0;
}
```

Các địa chỉ trong mảng 1 chiều

```
int a[] = {5, 3, 7, 5, 9, 1};
cout << "Dia chi mang: " << a << endl;
for(int i = 0; i < 6; i++)
{
    cout << i << "   " << &a[i] << endl;
}

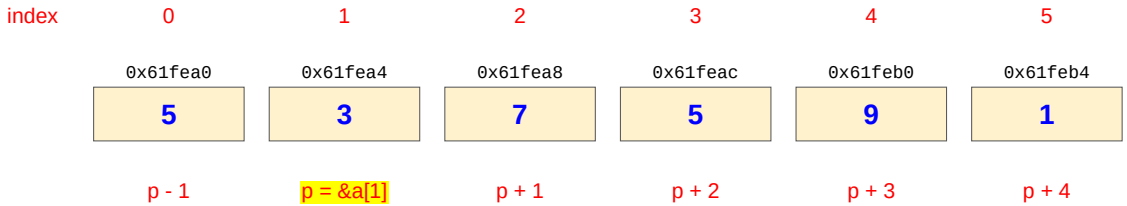
double b[] = {5.1, 3.6, 7.9, 5.3, 9.3, 1.9};
cout << "Dia chi mang: " << b << endl;
for(int i = 0; i < 6; i++)
{
    cout << i << "   " << &b[i] << endl;
}
```

Dia chi mang: 0x61fea0
0 0x61fea0
1 0x61fea4
2 0x61fea8
3 0x61feac
4 0x61feb0
5 0x61feb4

Dia chi mang: 0x61fe70
0 0x61fe70
1 0x61fe78
2 0x61fe80
3 0x61fe88
4 0x61fe90
5 0x61fe98

Con trỏ và mảng 1 chiều

```
int a[] = {5, 3, 7, 5, 9, 1};
```

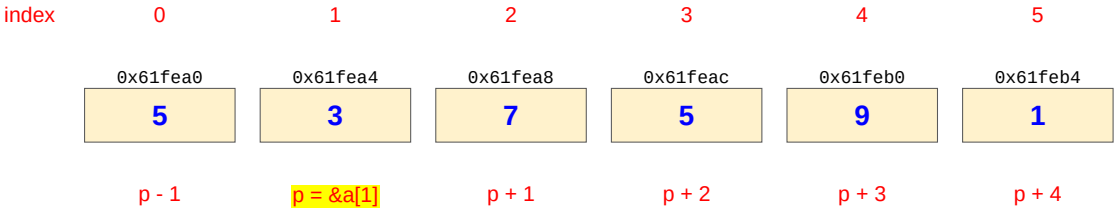


Phép cộng và trừ hai địa chỉ con trỏ

- Khi con trỏ cộng/trừ với một số nguyên, địa chỉ của con trỏ sẽ tăng/giảm lên một số lượng bộ nhớ tương ứng với số nguyên nhân vào kích thước của kiểu dữ liệu mà con trỏ đang trỏ tới.

Con trỏ và mảng 1 chiều

```
int a[] = {5, 3, 7, 5, 9, 1};
```



Ví dụ:

```
int a[] = {5, 3, 7, 5, 9, 1};

int* p = &a[1];

cout << *(p - 1) << endl;
cout << *(p + 1) << endl;
```

```
int a[] = {5, 3, 7, 5, 9, 1};
```




```
for(int i = 0; i < 6; i++)
{
    cout << a + i << endl;
}
```

```
for(int i = 0; i < 6; i++)
{
    cout << *(a + i) << endl;
}
```



Xuất mảng

- Hàm xuất mảng áp dụng kỹ thuật con trỏ

```
void xuat(int* a, int n)
{
    for(int i = 0; i < n; i++)
    {
        cout << *(a + i) << endl;
    }
}
```

```
int main()
{
    int a[] = {5, 3, 7, 5, 9, 1};
    int n = 6;

    xuat(a, n);

    return 0;
}
```



Khởi tạo giá trị mảng

- Hàm khởi tạo giá trị mảng áp dụng kỹ thuật con trỏ

```
void khoiTao(int* a, int& n)
{
    n = 6;
    *(a + 0) = 3;
    *(a + 1) = 1;
    *(a + 2) = 6;
    *(a + 3) = 9;
    *(a + 4) = 7;
    *(a + 5) = 8;
}
```

```
int main()
{
    int a[100];
    int n = 0;

    khoiTao(a, n);
    xuat(a, n);

    return 0;
}
```

Khởi tạo giá trị mảng

- Hàm khởi tạo giá trị mảng áp dụng kỹ thuật con trỏ

```
void nhap(int* a, int& n)
{
    cout << "So luong can nhap: ";
    cin >> n;
    for(int i = 0; i < n; i++)
    {
        cout << "\tNhap a[" << i << "] = ";
        cin >> *(a + i);
    }
}
```

```
int main()
{
    int a[100];
    int n = 0;

    nhap(a, n);
    xuat(a, n);

    return 0;
}
```

Cú pháp cấp phát bộ nhớ cho mảng 1 chiều

```
Kiểu_Dữ_Liệu* ten_mang = new Kiểu_Dữ_Liệu[kích_thước];
```

- Ví dụ:

```
int main()
{
    int* a = new int[100];
    int n = 0;

    khoiTao(a, n);
    xuat(a, n);
    return 0;
}
```

Tính tổng: áp dụng con trỏ mảng

Yêu cầu: Viết hàm tính tổng các giá trị trong mảng?

Phân tích:

- Input:
  - Mảng a => int* a
  - Số lượng n => int n
- Output:
  - Tổng các giá trị => return giá trị

```
int tinhTong(int* a, int n)
{
    int t = 0;
    for(int i = 0; i < n; i++)
    {
        t = t + *(a + i);
    }
    return t;
}
```



Bài tập nhỏ

```
#include <iostream>
using namespace std;
int main()
{
    int a[] = {1, 3, 5, 7, 9, 2, 4, 6, 8};
    cout << *(a + 3) + *(a + 7);
    return 0;
}
```

Nguồn: Đề thi

Khai báo

```
Kiểu_Cấu_Trúc* ten_bien;
```

Cấp phát

```
ten_bien = new Kiểu_Cấu_Trúc();
```

Vừa khai báo vừa cấp phát

```
Kiểu_Cấu_Trúc* ten_bien = new Kiểu_Cấu_Trúc();
```

Phương pháp

```
ten_bien->thuoc_tinh
```

- Ví dụ:

```
struct Diem
{
    int x;
    int y;
};
int main()
{
    Diem a = {3, 5};
    Diem* p = &a;

    cout << "Hoanh do x = " << p->x << endl;
    cout << "Tung do  y = " << p->y << endl;
    return 0;
}
```



Truy xuất thuộc tính

Minh họa

```
void xuat(Diem d)
{
    cout << "(" << d.x << "; " << d.y << ")\n";
}
```

```
Diem* b = new Diem;
b->x = 6;
b->y = 8;

xuat(*b);

Diem* c = new Diem;
(*c).x = 7;
(*c).y = 9;

xuat(*c);
```



Vừa cấp phát vừa khởi tạo

Minh họa

```
Diem* d = new Diem({1, 3});

xuat(*d);
```

Minh họa

```
void khoiTao(Diem* d, int& n)
{
    n = 3;
    *(d + 0) = {1, 2};
    *(d + 1) = {2, 6};
    *(d + 2) = {3, 1};
}
```

```
void xuatMang(Diem* d, int n)
{
    for(int i = 0; i < n; i++)
    {
        xuat(*(d + i));
    }
}
```

```
int main()
{
    Diem* arr = new Diem[100];
    int n = 0;

    khoiTao(arr, n);
    xuatMang(arr, n);

    return 0;
}
```

Minh họa

```
struct Node
{
    int data;
    Node* pNext;
};
```

```
struct NODE
{
    int data;
    NODE* pLeft;
    NODE* pRight;
};
```

BÀI 1

Xây dựng mảng 1 chiều lưu trữ các số thực. Sau đó, viết hàm thực hiện các chức năng sau:

1. Tạo giá trị ngẫu nhiên (số thực, 2 chữ số sau dấu chấm) thuộc khoảng (-268; 339). Số lượng thuộc đoạn [10; 20]
2. Xuất (in) mảng vừa tạo.
3. Tính tổng các giá trị có trong mảng.
4. Tính tổng, tích các giá trị có trong mảng (viết 1 hàm).
5. Đếm số lần xuất hiện 1 phần tử x bất kỳ.

BÀI 1 *(Tiếp theo)*

Xây dựng mảng 1 chiều lưu trữ các số thực. Sau đó, viết hàm thực hiện các chức năng sau:

6. Trả về các phần tử chẵn (chẵn-lẻ dựa vào giá trị nguyên của số thực) có trong mảng.
7. Tìm vị trí phần tử nhỏ nhất trong mảng.
8. Tạo một mảng đảo ngược.
9. Trả về các số âm trong mảng.
10. Trả về số lượng các giá trị trong mảng một chiều thuộc đoạn [x, y].
11. Trả về các giá trị trong mảng một chiều thuộc đoạn [x, y].

Bài 2

Áp dụng kỹ thuật cấp phát động cho mảng một chiều. Viết HÀM thực hiện các yêu cầu sau:

1. Tạo mảng số thực có các giá trị sau: | 0.8 | 5.6 | 9.1 | 7.3 | 10 | 5.9 | 7.2 | 9.3 | 8.0 | 8.7 |
2. Xuất mảng (in) ra màn hình.
3. Kiểm tra mảng có toàn dương.
6. Kiểm tra mảng có đối xứng.
7. Kiểm tra trong mảng có chứa số nguyên tố, số chính phương.
8. Tìm một giá trị có trong mảng.

***Bài 2** (tiếp theo…)*

9. Tìm các số nguyên tố có trong mảng.
10. Tìm các số chính phương có trong mảng.
11. Trả về các số âm trong mảng.
12. Tìm các số dương trong mảng và tính tổng của chúng.
13. Đảo ngược các phần tử trong mảng.

Bài 3

Viết chương trình quản lý và tính tiền lương cho các nhân viên trong một công ty. Nhân viên có các thông tin sau: mã nhân viên, họ tên, lương cơ bản, số ngày làm và lương hằng tháng.

Công thức tính tiền lương hằng tháng cho nhân viên như sau:

Lương hằng tháng = lương căn bản + số ngày làm việc * 180.000 đ

Nếu lương hằng tháng hơn 8.000.000 thì thưởng thêm 5%.

Nếu lương cơ bản dưới 5.000.000 thì phụ cấp thêm 10%.

Áp dụng cấp phát động. Xây dựng chương trình đáp ứng các yêu cầu sau:

1. Tạo dữ liệu các nhân viên dùng để kiểm thử (10 nhân viên)
2. Xuất danh sách nhân viên trong công ty.

Bài 3 *(tiếp theo…)*

Chương trình đáp ứng các yêu cầu sau:

3. Tính và cập nhật tiền lương cho các loại nhân viên.
4. Tính tổng lương hằng tháng mà công ty phải trả cho toàn bộ nhân viên.
5. Tính tổng lương hằng tháng các nhân viên có lương cơ bản < 5 triệu.
6. Tìm nhân viên theo theo mã nhân viên.
7. Tìm các nhân viên có lương hằng tháng thấp nhất.
8. Tìm các nhân viên có lương cơ bản cao nhất cao nhất.


Thank you

Hỏi - Đáp

